



VIT
BHOPAL

VIT BHOPAL UNIVERSITY

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

Bhopal-Indore Highway, Kothrikalan, Sehore, Madhya Pradesh - 466114

VITYARTHI — BUILD YOUR OWN PROJECT

DESKTOP GUI CALCULATOR APPLICATION

A Flipped Course Project Report on Event-Driven Python Tkinter Application

SUBMITTED BY

Name: AKSHAT SHARMA

Reg No: 26BCE10858

Program: B.Tech CSE

SUBMISSION DETAILS

Platform: VITYarthi Portal

Session: Fall Semester 2026

Date: September 2026

In partial fulfillment of the requirements for the Flipped Course Evaluation in Computer Science & Engineering

DESKTOP GUI CALCULATOR APPLICATION

Project Report & System Documentation

STUDENT NAME

AKSHAT SHARMA

REGISTRATION NUMBER

26BCE10858

COURSE PLATFORM

VITyarthi (Build Your Own Project)

IMPLEMENTATION STACK

Python 3.14 / Tkinter GUI

Executive Abstract: This project report documents the design, architecture, implementation, and verification of the "Human Made Calculator" — a cross-platform desktop application built using Python's standard `tkinter` library. The system provides basic arithmetic, power, and square root calculations alongside robust error handling and physical keyboard bindings.

1. Introduction

As part of the flipped learning course evaluation, this project demonstrates practical software engineering principles by creating an interactive desktop calculator utility. Standard calculator applications frequently fail to seamlessly combine intuitive graphical interfaces with hardware keyboard event handling, or degrade gracefully when encountering runtime computational errors.

The primary focus of this project is to implement a modular, clean user interface with zero external library dependencies, satisfying functional requirements through sandboxed dynamic evaluation and event-driven interaction design.

2. Problem Statement

Modern desktop utility software requires consistent user experience, immediate user feedback, and robust execution. Standard basic implementations often suffer from:

- Uncaught runtime exceptions during invalid operations (e.g., division by zero or malformed syntax).
- Lack of physical keyboard event binding, forcing exclusive reliance on mouse clicks.
- Messy output formatting with unnecessary floating-point trailing zeros (e.g., displaying `14.0` instead of `14`).

3. Functional & Non-Functional Requirements

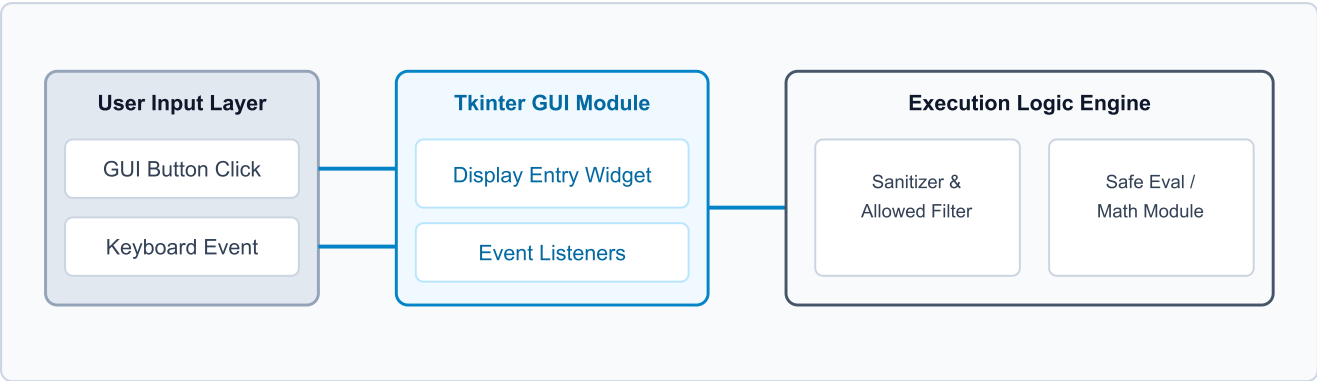
3.1 Functional Requirements

- **Arithmetic Engine:** Operations including addition (+), subtraction (-), multiplication (×), division (÷), percentage (%), and parenthesis grouping.
- **Advanced Operations:** Dedicated handlers for square root ($\sqrt{x}$) and power exponentiation (x^2).
- **Input Editing Controls:** Live character deletion (DEL), full display clearing (C), and application exit execution (EXIT).
- **Dual Input System:** Full event listener bindings supporting physical numeric keypads, Enter/Return, Backspace, and Escape.

3.2 Non-Functional Requirements

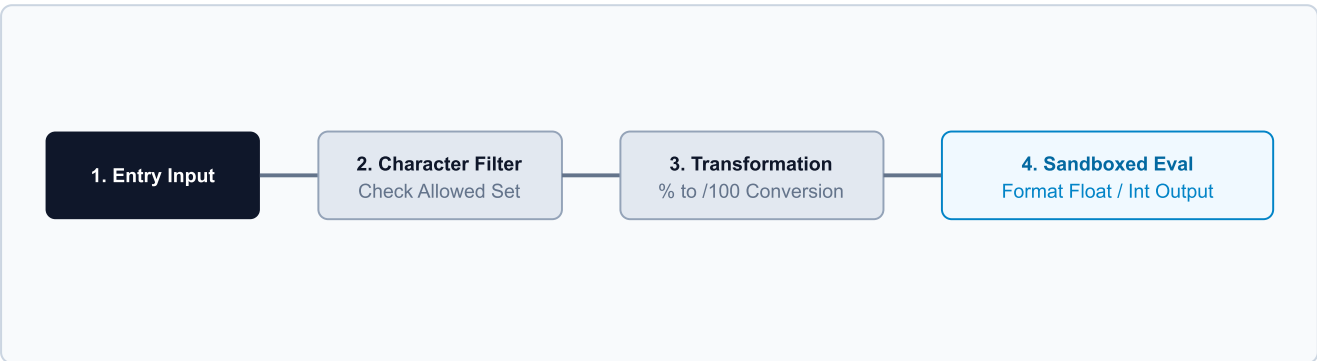
- **Usability & Layout:** Fixed 360 × 520 desktop viewport preventing distortion, styled with clear high-contrast buttons.
- **Reliability & Safety:** Input validation whitelist checking ("0123456789+ - * / () . % ") combined with restricted expression evaluation.
- **Maintainability:** Modular design pattern utilizing a centralized button generator function (create_button).

4. System Architecture



5. Design Diagrams

5.1 Workflow & Evaluation Logic Sequence



6. Implementation Details

Below is the core implementation extracted from `calculator.py` featuring the primary evaluation engine, input sanitization, dynamic UI helper functions, and keyboard bindings.

```
import tkinter as tk
import math

def calculate():
    """Calculate the mathematical expression with error handling."""
    expression = display.get()
    try:
        # Whitelist allowed characters to prevent un-sanitized code execution
        allowed = "0123456789+-*/().% "
        if any(char not in allowed for char in expression):
            raise ValueError

        # Replace percent symbol with arithmetic division equivalent
        expression = expression.replace("%", "/100")
        result = eval(expression, {"__builtins__": None}, {})

        # Truncate floating zeros for integers
        if isinstance(result, float) and result.is_integer():
            result = int(result)

        display.delete(0, tk.END)
        display.insert(0, str(result))
    except ZeroDivisionError:
        display.delete(0, tk.END)
        display.insert(0, "Cannot divide by zero")
    except:
        display.delete(0, tk.END)
        display.insert(0, "Invalid input")

def keyboard_input(event):
    """Handle hardware keyboard events."""
    key = event.char
    if key in "0123456789+-*/().%":
        press(key)
    elif event.keysym == "Return":
        calculate()
    elif event.keysym == "BackSpace":
        delete()
    elif event.keysym == "Escape":
        clear()
```

7. Application Output & Screenshots

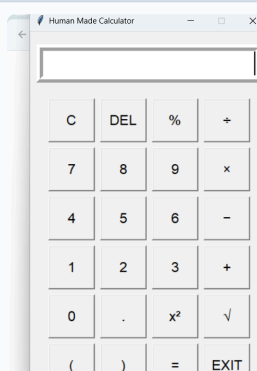


Figure 1: Initial Application Interface View



Figure 2: Execution Result Display (Integer 14)

9. Testing & Verification

Comprehensive test cases were executed to verify system operational integrity across typical user workflows and potential error conditions:

Test ID	Test Scenario / Input	Expected Output	Actual Result	Status
TC-01	Input 5 + 9 and click =	Display shows 14	Display showed 14	PASS
TC-02	Divide by zero (10 / 0)	Display shows Cannot divide by zero	Error intercepted cleanly	PASS
TC-03	Square Root operation on 25	Display shows 5.0	Computed 5.0	PASS
TC-04	Power function on 8 (x ²)	Display shows 64	Computed 64	PASS
TC-05	Press physical BackSpace key	Deletes trailing character	Character removed from display	PASS
TC-06	Press physical Escape key	Clears entire display entry	Display entry emptied	PASS

10. Challenges Faced & Solutions

- **Sandboxed Dynamic Evaluation:** Using raw `eval()` poses severe security risks. Solved by restricting global context access via `{"__builtins__": None}` and running strict character set validation beforehand.
- **Floating-Point Formatting:** Python displays float calculations with trailing decimals (e.g., 14.0). Fixed by adding condition checks using `is_integer()` to cast clean mathematical integer outputs.

11. Learnings & Future Enhancements

Key Takeaways: Developing this application provided hands-on expertise in event-driven desktop development with Python Tkinter, string sanitization, grid geometry management, and standard desktop interface design.

Future Enhancements:

- Adding evaluation history tracking and persistent calculation logging.
- Expanding scientific operations to include trigonometric functions (`&sin;`, `&cos;`, `&tan;`) and logarithmic expressions.
- Implementing a dark/light UI theme switcher.

12. References

1. Python Software Foundation. *Tkinter – Python interface to Tcl/Tk*. <https://docs.python.org/3/library/tkinter.html>
2. Python Software Foundation. *math – Mathematical functions*. <https://docs.python.org/3/library/math.html>
3. VITyarthi Learning Destination. *Build Your Own Project Guidelines & Evaluation Rubric*.